# Troubleshooting

Start with the symptom, confirm the relevant manager and asset convention, then compare the smallest focused demo.

## Table of contents



## Use diagnostics and tests first

Manager components expose a log-level threshold:

- `Critical` shows only `Critical` messages, such as missing data or configuration errors.
- `Warning` also shows rejected operations, such as a `Try...()` call returning `false` .
- `Verbose` includes routine successful operations as well.

If a property-drawer warning visible in the Inspector is absent at runtime, that is expected: Inspector validation does not replace runtime validation of remotely loaded or generated data.

The current source includes Edit Mode tests for BigNumber/math, wrappers, drop tables, and legacy serialization. Play Mode coverage includes:

- Businesses, currencies, wrappers, merges, caps, groups, rounds, costs, and totals.
- Drop tables, upgrades, contextual filters, overrides, and queues.
- Boosts, managers, locations, locks, offers, shops, daily rewards, achievements, progression, prestige, and save/load.
- Offline profit and production modifier behavior.

Reproduce a failure in the closest focused scene, run the nearest existing tests, then add a project-side regression test before changing behavior. For import or compiler failures that prevent tests from running, follow Compilation errors after import or update ( `CompilationErrors.pdf` ).

## A demo path cannot be found

Documentation breadcrumbs start at the relative package root `EasyIdleGame` , regardless of where that folder is installed under `Assets` . The package may be moved through Unity as one complete folder. Preserve its internal structure and every `.meta` file so assemblies and GUID-backed scene, prefab, and asset references remain intact. `Resources.Load` and `Resources.LoadAll` continue to work because they depend on folders named `Resources` , not the package root.

Focused-demo readmes contain explanatory content and resource prefixes only; open demo scenes from the package-relative locations in Demo catalog ( `06-demo-catalog.pdf` ). Shared demo panel settings are serialized directly in the scenes and do not use a package-path fallback.

Two external workflow limitations remain:

- The existing Play Mode save/load tests create temporary `Resources` assets at their original test location. Those tests are intentionally unchanged and are not relocation-safe.
- Importing or reimporting an archive can restore its original import location instead of updating a relocated copy. Remove or reconcile the previous installation first so the project never contains two package versions.

Do not move the folder outside `Assets`; Unity will not import its scripts or assets there.

## A manager Instance is missing

- Put one enabled component in the active scene.
- Add `CurrencyManager` and `BusinessesManager` before any code accesses their required singletons.
- Add optional managers for every optional definition used by outputs or save data.
- Add every manager referenced by a configured lock. Lock evaluation throws an `InvalidOperationException` naming the missing manager when the scene is misconfigured.
- Avoid duplicate manager instances in additive scenes.
- Ensure initialization code does not run before the scene manager GameObject is active.

An output that grants a boost or manager needs the corresponding optional manager; otherwise that target is not applied.

## A saved asset cannot be restored

The loader uses definition asset names and `Resources.LoadAll`.

Check that:

- The asset is below a Resources folder.
- Its name has not changed since the save was written.
- No same-type Resources asset has the same name.
- The optional manager for that holder type exists before load.
- Old JSON fields are still compatible with the current serializable holder.

Use stable machine-oriented asset names and separate player-facing names in `Metadata`.

## The save file is not where expected

Log `SaveAndLoad.Path`, not just `Application.persistentDataPath`.

The path is built with `Path.Combine`, and saves written by earlier versions to the legacy concatenated location are migrated automatically on `Awake()` / `Load()`.

## A business will not produce

Check all of these:

- Current business amount is greater than zero.
- Manual input calls the correct displayer/holder production method, or `autoProduce` is enabled.
- If automation depends on a manager, the manager is owned/unlocked at `autoProduceOnLevel`.
- An active `AutoProduceBoost` targets the effective business groups if no manager is available.
- Production inputs are affordable.
- Production lower and upper locks pass.
- `timeToProduce` is positive.
- The active location and group targets are correct.

For scaled production cost, enable partial production only if a smaller affordable batch should start.

Unlock state and permission to produce are separate checks: an unlocked business can still be blocked by its production locks and upper production locks.

## A business produces but nothing is received

- Confirm at least one output table contains a valid target.
- For Independent tables, confirm `dropChance` is above zero.
- For Weighted Random tables, confirm entries have positive weight and `totalDropAmount` is at least one.
- Check output amount/minimum and maximum values.
- If `autoCollect` is disabled, inspect `BusinessHolder.unclaimedOutputs` and wire a claim action.
- Confirm the target's manager exists.
- Check business/currency caps and typed group capacity.
- For a consumable producer, confirm the source copies are expected to disappear.

## Buying maximum or percentage fails

Maximum/percentage calculations need a non-empty cost. `Business.GetMaxBuyableAmount` throws for an empty cost list.

Also check:

- `maxBuyableAmount` purchase cap.
- `maxAmount_` ownership cap.
- Active business upgrade-level cap.
- `maxBusinessAmount` override upgrades.
- Shared business-group capacity.
- Currency/business affordability after contextual cost modifiers.

Use `PurchasePreview` for UI instead of performing a trial mutation.

## An upgrade or boost affects the wrong target

- Replace legacy string groups with typed groups.
- Confirm the source business and target business are not being confused in `UpgradeBlockFilter`.
- Verify target currency/business/upgrade/boost and group lists.
- Check `SourceType` propagated by the operation.
- Check the runtime mask; manual active contexts contain both Active and Manual flags.
- Confirm the business level is within the configured inclusive range.
- Inspect `Business.ignoredMultipliers` for production opt-outs.
- For boost merging, compare the actual group lists and upgrade blocks.

Empty normal target-group lists are global. A non-empty filter only matches when its context contains the required target.

## A timed upgrade does not apply immediately

This is expected when `upgradeDurationInSeconds > 0`.

- Inspect `UpgradeHolder.pendingUpgrades`.
- Confirm `UpgradesManager.Update` is running.
- In Sequential mode, later entries begin after the previous queued end time.
- Listen to `OnUpgradeApplied`, not only the purchase event.
- Use `SkipUpgradeWait(upgrade)` only when the design deliberately completes all pending entries for that upgrade.

## Offline profit looks different from live play

The calculator uses at most 30 simulation segments and average/optimistic production paths for bounded offline calculation.

Compare:

- Elapsed duration after `maxOfflineSeconds` clamping.
- `simulateBoosts` and manager simulation choices.
- Active/offline contextual filters.

- Production inputs and business chains.
- Caps and group capacity.
- Stockpile versus auto-collection.
- Production modifier registration.
- Time skip calls that may have already applied the same period.
- Active boost/manager timer simulation; timers advance once per simulation segment regardless of the number of businesses.

Use `ProfitApplicationSummary` to distinguish calculated output from the amount accepted after caps.

## A daily reward cannot be claimed or has unexpected outputs

Daily reward days are elapsed reward-cycle days, not calendar dates. `GetCurrentDay()` calculates the day from `initDate` and `dayLengthHours`; changing either value changes which day is current. Claiming an unclaimed past day is intentionally supported, but claiming a future day is rejected.

`TryClaimDailyRewards` returns `false` and does not add the day to `claimedDays` when:

- Non-positive days.
- Days after `GetCurrentDay()`.
- Days already present in `claimedDays`.
- No valid exact-day or repeating `Reward` is configured for the requested day.

For unexpected reward contents, check all matching rules:

- Every exact-day entry with the requested `day` and a non-null `Reward` is combined when `combineExactDayRewards` is `true`, which is the default. Set it to `false` only for legacy first-match behavior.
- Every repeating entry whose positive `level` evenly divides the requested day is also included. Entries with a non-positive `level` or null `Reward` are skipped.
- `scaleEachDayReward` multiplies repeating reward output amounts by the requested day number.
- `GetDailyRewards` is an average preview, while `TryClaimDailyRewards` performs the actual random drop rolls. A random reward can therefore differ from its preview, including producing no outputs; a configured reward is still claimed and the day is recorded.

## Shop rewards are missing or duplicated

All built-in purchase paths grant reward tables immediately and add shop holder amount:

- Ad/IAP success: call `CompleteExternalPurchase` once after verification.
- In-game/free normal purchase: `TryBuyItems` routes through `ForceBuyItemsForFree`, which calls the same completion logic.
- `TryConsumeItem` removes holder amount and grants the reward tables a second time.

Do not consume a completed entitlement unless two reward grants are intended. Confirm only one payment method is configured; otherwise Ad takes priority, then Real Money, then In-Game Cost. Bulk completion amounts are floored, rejected when non-positive, and clamped to the remaining purchase limit; the reward count always matches the amount actually added.

## Prestige resets too much or too little

Review both holder reset flags and unlock-state reset flags. They are independent.

Then check:

- Explicit exclusions.
- Typed-group exclusions.
- Per-call `PrestigeResetOptions`.
- Currency field `exludedCurrencies` with its compatibility spelling.
- Shop ownership, which is not reset by the current prestige manager.
- Location starter outputs and the default active location.

Inspect the returned `ResetSummary` instead of inferring the reset from UI state.

## Demo UI looks wrong

- Import TextMeshPro essentials.
- Verify font/material references.
- Apply the intended outline value ( `0.42` ) when matching the demo style.
- Preview at `9:16` or taller portrait aspect.
- Use **Tools > EasyIdleGame > RedrawAll** for editor previews.
- Confirm `EasyIdleGame.UI` resolves TextMeshPro and uGUI.